

Race master 4D: racist game

Prak documentation

Michael Gustavo & Danila Malyshev

September 67, 2026

1 Introduction

This project is a small C++ program for viewing traffic at a four-way road intersection. It uses SFML to draw the simulation and ImGui for the control panel. The main idea is to see how traffic lights, cars, and pedestrians behave when the signal timings or traffic settings are changed.

The intersection has roads in the north, south, east, and west directions. Each road has six lanes: three for incoming traffic and three for outgoing traffic. The incoming lanes have fixed movements: right turn, straight ahead, and left turn. The program is a simplified model, not a real traffic forecast.

2 How the simulation works

The **World** class builds the roads, traffic lights, and crosswalks. A **Road** contains six **Lane** objects. Lanes hold cars and their waypoints. **Car** and **Pedestrian** are moving objects based on the common **Entity** class. The **app** code opens the window, updates the world, and draws the objects.

The intersection is in the middle of the world, and the model uses 8 pixels for one metre. A car follows the waypoints for its lane, keeps space from the car in front, and stops for a red signal. The car only reacts to the signal when it is inside the selected visibility distance. On yellow, it can continue if it is already close to the stop line. Left-turning cars can wait for opposing traffic, and cars also wait if there is no room in their exit lane. Collisions are not simulated.

The traffic signals have phases for north–south straight/right traffic, then north–south left turns, followed by the same movements for east–west traffic. Each vehicle phase has a green and a yellow part. The last phase stops all cars while pedestrians use the crosswalks.

There are two signal modes. In static mode, green, yellow, and the all-red pedestrian phase use values selected in the control panel. The default values are 12, 3, and 12 seconds. The pedestrian phase can last longer if somebody is still crossing.

In automatic mode, green time depends on the queue. The target is $\min(30, 5 + 1.5n)$ seconds, where n is the number of cars waiting for the active movement. The controller can end an empty green phase after five seconds if there is traffic waiting on the other road. Automatic yellow stays at 3 seconds. Pedestrian timing also responds to the waiting queue, with a target of $\min(24, 5 + 1.5p)$ seconds, where p is the number of waiting pedestrians.

3 Control panel

The ImGui panel contains these settings:

Setting	What it does
Signal mode	Selects static or automatic timing.
Green duration	Static green time: 1–90 seconds; default 12.
Yellow duration	Static yellow time: 1–15 seconds; default 3.
Red / walk duration	Static all-red pedestrian phase: 1–90 seconds; default 12.
Signal visibility	Distance for reacting to a signal: 5–100 metres; default 30.
Car speed range	Range for new cars: 5–160 km/h overall; default 30–120 km/h.
Car arrival interval	Minimum: 1–30 seconds; maximum: from the minimum up to 60 seconds; default 4–8 seconds.

The static timing sliders only apply in static mode. The other settings can be changed while the simulation is running. A new car gets a speed limit sampled from the selected range, and starts at a speed between the minimum and its own speed limit.

When the world starts, it tries to place one car on each road. After that, one random interval is selected for each cycle. At the end of the interval, the program tries to add one car to every road, so all roads use the same period. If there is no room on a road, that arrival is skipped. The **Add car** button tries to add one car to each incoming lane with room, and **Next background** changes the background image.

4 Files and running the program

The main files are:

- **main.cpp** starts the application.
- **app.cpp** contains the window loop, control panel, and drawing.
- **app.h** contains application declarations and rendering helpers.
- **objects.h** contains the simulation classes and most of the model code.
- **objects.cpp** includes the model header for the CMake build.
- **CMakeLists.txt** describes the C++ project and its dependencies.

The project uses CMake 3.24 or later, C++17, SFML 3, OpenGL, and ImGui-SFML. With these dependencies installed, it can be built from a terminal with:

```
cmake -S crossroads -B crossroads/build
cmake --build crossroads/build
```

The background manager looks for images under `../assets/`, but those images are not included in the **crossroads/** source folder. The roads and traffic can still be drawn by the simulation.